functionalities the consumer doesn’t care about should be covered by other types of tests from the provider side.

6.1.2 Using provider states effectively

As we mentioned in chapter 4, section 4.1.4, contract tests should be isolated and run without any dependencies because they sit above unit tests. However, what if a contract test requires some data that already exists on the provider? In some cases, you don’t need to worry about this, but there are scenarios where consumers require an interaction in which the provider needs to be in a certain state. For example, in our web application and movies API scenario, the web application might send a `GET` request to the `/movie/2` endpoint to retrieve a specific movie. For the contract test to be stable, the data provider should always be in the same state of having the correct data set up before this interaction is verified. This step is crucial to avoid flaky contract tests. Imagine if the data provider was reset, and a contract test was run without ensuring the data provider was in the right state. The contract test will fail, not because of actual problems between the consumer and provider contract, but because the provider wasn’t in the right state.

To ensure that providers are set up correctly before verifying the contract, contract testing tools, such as Pact, introduced the concept of provider states. Provider states allow data providers to define the state they need to be in to verify the interaction from the consumer contract successfully.

When you write a provider contract test with provider states, you need to also make sure that the provider state is provided from the consumer. In a nutshell, you follow this approach:

1. Set up the consumer contract test with a provider state.
2. Define the state on the provider.

This can also be visualized in the following code. Don’t worry about the details of the code, but just pay attention to the flow of the test on both consumer and provider: